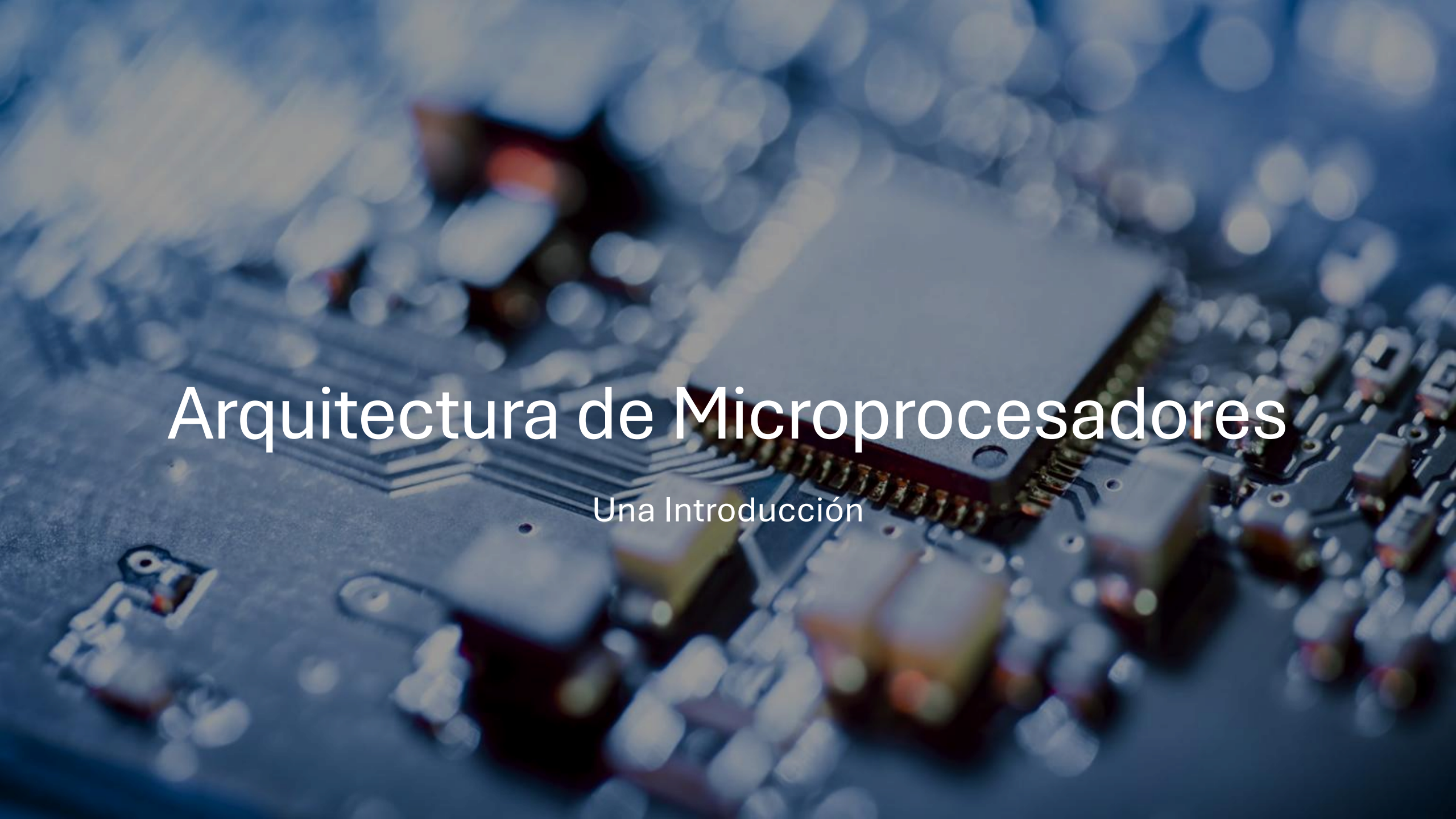


A close-up, shallow depth-of-field photograph of a microprocessor circuit board. The central focus is a large, square, silver-colored microprocessor chip with gold-plated pins. Surrounding it are various other components, including smaller chips, capacitors, and resistors, all mounted on a dark green printed circuit board. The background is heavily blurred, creating a bokeh effect with soft, out-of-focus light spots.

Arquitectura de Microprocesadores

Una Introducción

¿Qué es un microprocesador?

Un microprocesador es el cerebro de un ordenador. Es un circuito integrado extremadamente complejo que ejecuta las instrucciones de un programa informático. Imagina que es un director de orquesta, coordinando miles de millones de operaciones por segundo para llevar a cabo las tareas que le pedimos a nuestro ordenador.

A large orange circle on the left side of the slide, partially cut off by the edge.

¿Qué es la
arquitectura de un
microprocesador?

La **arquitectura de un microprocesador** es el diseño y la organización interna de los componentes de un procesador, que determina cómo ejecuta las instrucciones y cómo interactúa con el resto del sistema (memoria, buses, periféricos, etc.). Este diseño incluye aspectos clave como el conjunto de instrucciones (ISA), el número y tipo de registros, la organización de la memoria y la forma en que se manejan las instrucciones, entre otros.

Four blue curved lines of varying lengths and orientations, located in the bottom right corner of the slide.

Componentes principales de la arquitectura de un microprocesador



Unidad Central de Procesamiento (CPU)

Unidad Aritmético-Lógica (ALU): Ejecuta operaciones matemáticas y lógicas.

Unidad de Control (CU): Dirige el flujo de instrucciones y controla el funcionamiento de otras unidades.

Conjunto de Instrucciones (ISA)

Define las operaciones que el procesador puede realizar, cómo se codifican y cómo interactúa con la memoria.

Un ISA es como un diccionario que define todas las palabras que un microprocesador puede entender y ejecutar. Estas "palabras" son las instrucciones básicas que el procesador puede realizar, como sumar números, mover datos de un lugar a otro o tomar decisiones lógicas.

Características principales de un ISA:

- **Tipos de datos:** Define los tipos de datos que el procesador puede manipular (enteros, números de punto flotante, caracteres, etc.).
- **Instrucciones:** Especifica las operaciones que el procesador puede realizar (aritméticas, lógicas, de control de flujo, etc.).
- **Modos de direccionamiento:** Describe las diferentes formas en que el procesador puede acceder a los datos en la memoria.
- **Registros:** Define los registros internos del procesador, que se utilizan para almacenar temporalmente los datos durante la ejecución de las instrucciones.

Registros ¿Qué son?

Imagina que un microprocesador es un cerebro y los registros son su memoria a corto plazo. Son pequeñas áreas de almacenamiento dentro del procesador, extremadamente rápidas, que se utilizan para guardar datos que se están procesando activamente.

Registro de 32 bits de propósito general

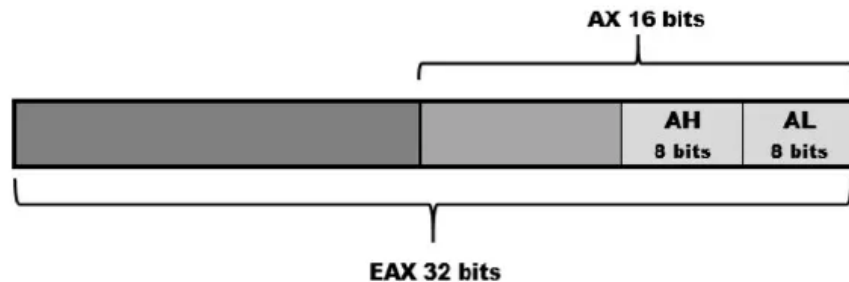
EAX
EBX
ECX
EDX
EBP
ESP
ESI
EDI

Registros de segmento de 16 bits

CS	ES
SS	FS
DS	GS

EFLAGS
EIP

Los registros de propósito general son los que se utilizan en operaciones aritméticas o mover direcciones de datos de un registro a otro. Así también cada registro puede dividirse en registros de 16 bits, un ejemplo:



¿Para qué sirven los registros?



Almacenamiento temporal: Los registros guardan los datos que se están usando en una operación matemática, una comparación lógica o cualquier otra tarea que el procesador esté realizando.



Aceleración del procesamiento: Al estar dentro del procesador, los registros son mucho más rápidos de acceder que la memoria principal (RAM). Esto significa que las operaciones que involucran datos en los registros se ejecutan mucho más rápido.



Dirección: Algunos registros se utilizan para almacenar direcciones de memoria, lo que permite al procesador acceder a datos específicos en la memoria principal.

Tipos de registros



Registros de propósito general:

Se utilizan para almacenar una amplia variedad de datos y pueden ser utilizados para diferentes propósitos.



Registros de propósito

específico: Están diseñados para realizar tareas específicas, como el contador de programa (que almacena la dirección de la siguiente instrucción a ejecutar) o el registro de estado (que almacena información sobre el estado actual del procesador).



Registros de datos: Almacenan los datos que están siendo procesados.



Registros de direcciones:

Contienen las direcciones de memoria donde se encuentran los datos.

Memoria caché

Imagina que tienes una biblioteca enorme. Cada vez que buscas un libro, tienes que recorrer todas las estanterías. Sería mucho más rápido si tuvieras una pequeña estantería cerca de tu escritorio con los libros que consultas más a menudo. Esa estantería pequeña es como la memoria caché en una computadora.

La memoria caché es una pequeña cantidad de memoria muy rápida ubicada dentro del procesador o cerca de él. Su función principal es almacenar temporalmente los datos y las instrucciones que se utilizan con más frecuencia, para que el procesador pueda acceder a ellos más rápidamente.

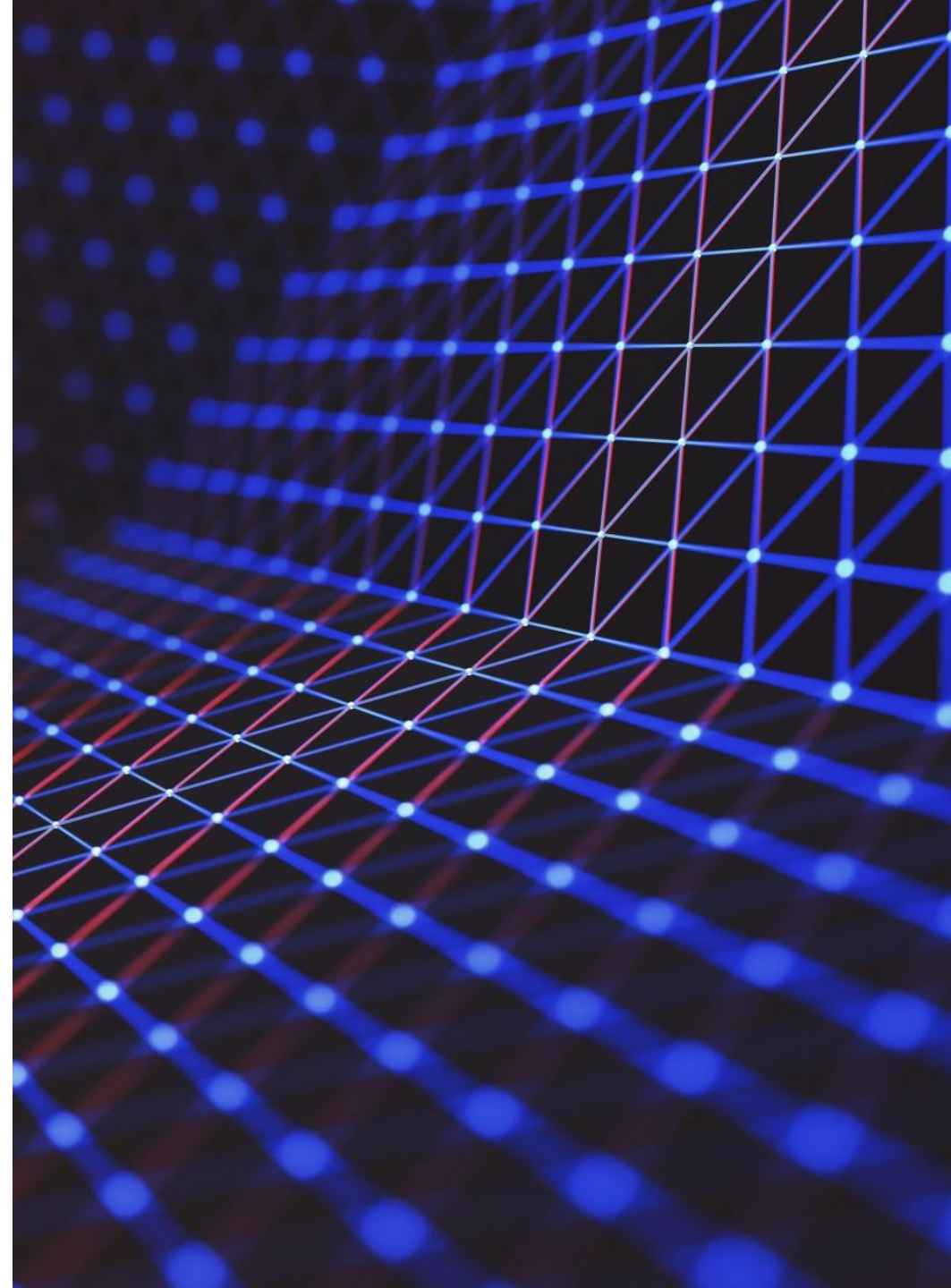


¿Por qué es importante la memoria caché?

Velocidad: La memoria caché es mucho más rápida que la memoria RAM (la memoria principal). Esto significa que el procesador puede acceder a los datos almacenados en el caché mucho más rápido que si tuviera que buscarlos en la RAM.

Rendimiento: Al reducir el tiempo que el procesador tarda en acceder a los datos, la memoria caché mejora significativamente el rendimiento general del sistema.

Reducción del cuello de botella: La memoria RAM, aunque es más rápida que el disco duro, sigue siendo un cuello de botella en el rendimiento de un sistema. La memoria caché ayuda a aliviar este cuello de botella.





Tipos de memoria caché

- **L1 (Nivel 1):** Es la más pequeña y rápida. Se encuentra integrada directamente en el núcleo del procesador.
- **L2 (Nivel 2):** Es más grande que la L1 y un poco más lenta. Suele ser compartida por todos los núcleos del procesador.
- **L3 (Nivel 3):** Es la más grande y la más lenta de las tres. Suele ser compartida por todos los núcleos del procesador.

Pipeline

¿Qué es un Pipeline de Instrucciones?

- Imagina una fábrica de automóviles. Cada coche pasa por diferentes estaciones de trabajo: una para la carrocería, otra para el motor, otra para la pintura, etc. Cada estación trabaja en un coche diferente al mismo tiempo, lo que permite producir muchos coches en menos tiempo.
- Un pipeline de instrucciones en un procesador funciona de manera similar. En lugar de coches, procesa instrucciones. En vez de estaciones de trabajo, tiene diferentes etapas.



¿Cómo funciona?

1. Fases del Pipeline

- I. **Fetch:** La instrucción se busca en la memoria y se trae al procesador.
- II. **Decode:** Se decodifica la instrucción para entender qué operación debe realizar.
- III. **Execute:** Se realiza la operación especificada en la instrucción.
- IV. **Memory:** Si la instrucción necesita acceder a la memoria (por ejemplo, para leer o escribir datos), se realiza esta acción.
- V. **Write back:** El resultado de la operación se escribe en el registro o la memoria.



¿Cómo
funciona?



2. Funcionamiento Simultáneo

- Mientras una instrucción está en la fase de ejecución, otra puede estar siendo decodificada, y otra más puede estar siendo buscada. Es como si hubiera varias instrucciones en diferentes etapas de procesamiento al mismo tiempo.

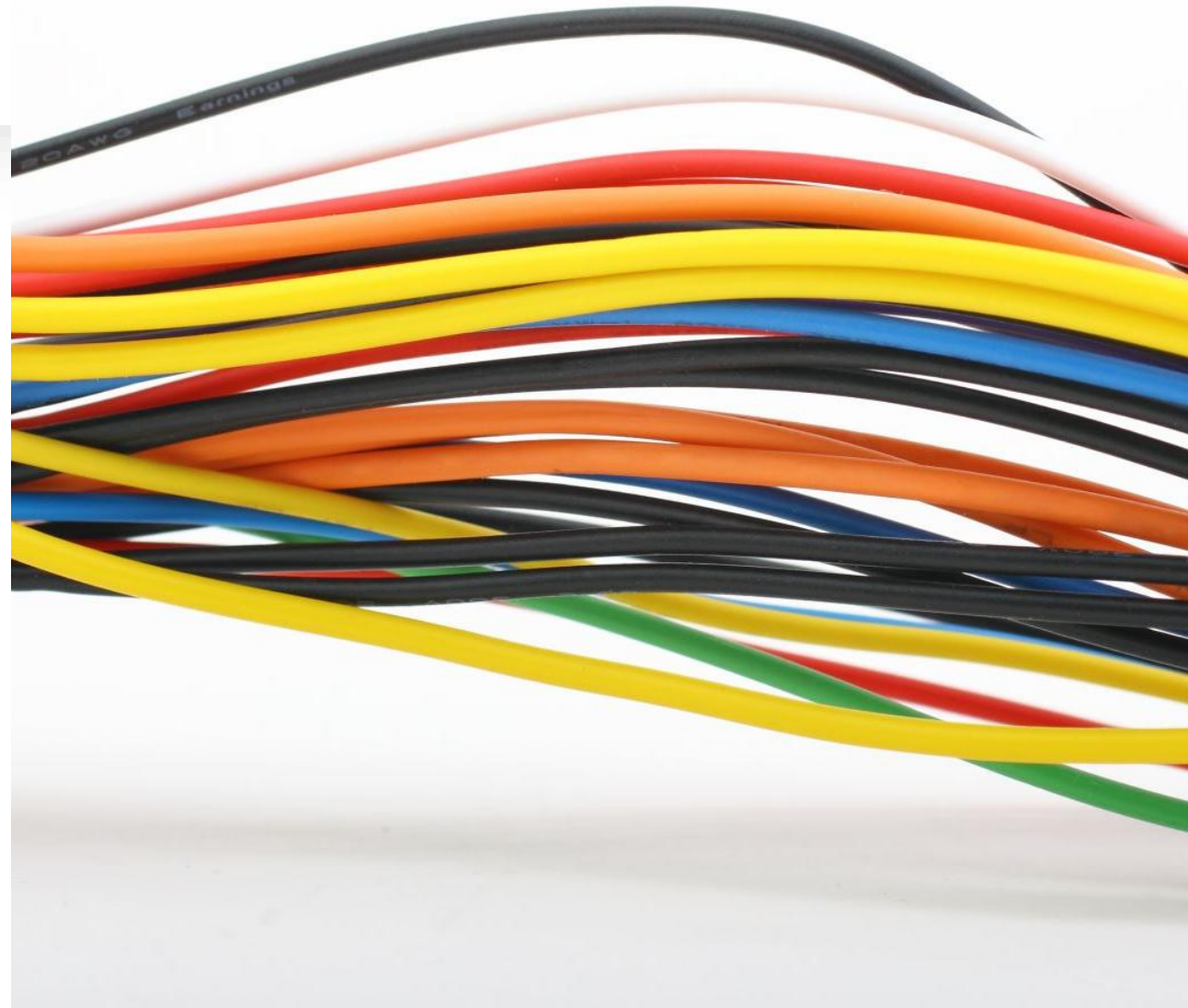
Bus de Datos y Direcciones

- Imagina tu computadora como una ciudad muy organizada. Para que toda la información fluya correctamente entre los diferentes edificios (procesador, memoria, dispositivos), se necesitan calles bien definidas. Esas calles son los **buses**.



¿Qué son los buses?

Los buses son conjuntos de cables que conectan los diferentes componentes de una computadora. A través de ellos circulan datos, direcciones y señales de control.



Existen tres tipos principales de buses

Bus de datos: Es como una carretera de doble sentido por la que viajan los datos. Sirve para transportar la información entre los distintos componentes. Por ejemplo, cuando quieres cargar una imagen, los datos de la imagen viajan por el bus de datos desde el disco duro hasta la memoria y luego al procesador.



Bus de direcciones: Es como una calle de un solo sentido que indica a dónde se deben enviar los datos. El procesador utiliza el bus de direcciones para especificar la ubicación exacta en la memoria donde quiere leer o escribir datos.



Bus de control: Es como un semáforo que controla el tráfico en las otras dos calles. El bus de control envía señales para sincronizar las operaciones de los diferentes componentes y coordinar la transferencia de datos.

¿Cómo funcionan juntos?

-
- **Ejemplo:** Imagina que quieres cargar un archivo de texto en un programa de edición. El procesador envía una dirección de memoria específica al bus de direcciones.

En resumen:

- **Bus de datos:** Transporta la información.
- **Bus de direcciones:** Indica dónde se encuentra la información.
- **Bus de control:** Coordina las operaciones.

Unidad de gestión de memoria (MMU)

Imagina que tienes un libro muy grande de recetas, pero solo puedes tener unas pocas páginas abiertas a la vez. La MMU es como un índice muy inteligente que te ayuda a encontrar la página correcta rápidamente, incluso si el libro es mucho más grande que la mesa donde estás trabajando.

En una computadora, la MMU es un componente de hardware que se encarga de traducir las direcciones de memoria que utiliza el software (direcciones virtuales) a las direcciones reales de la memoria física.



¿Por qué necesitamos una MMU?

Protección de memoria: La MMU evita que un programa acceda a áreas de memoria que no le pertenecen, lo que previene conflictos y mejora la estabilidad del sistema.

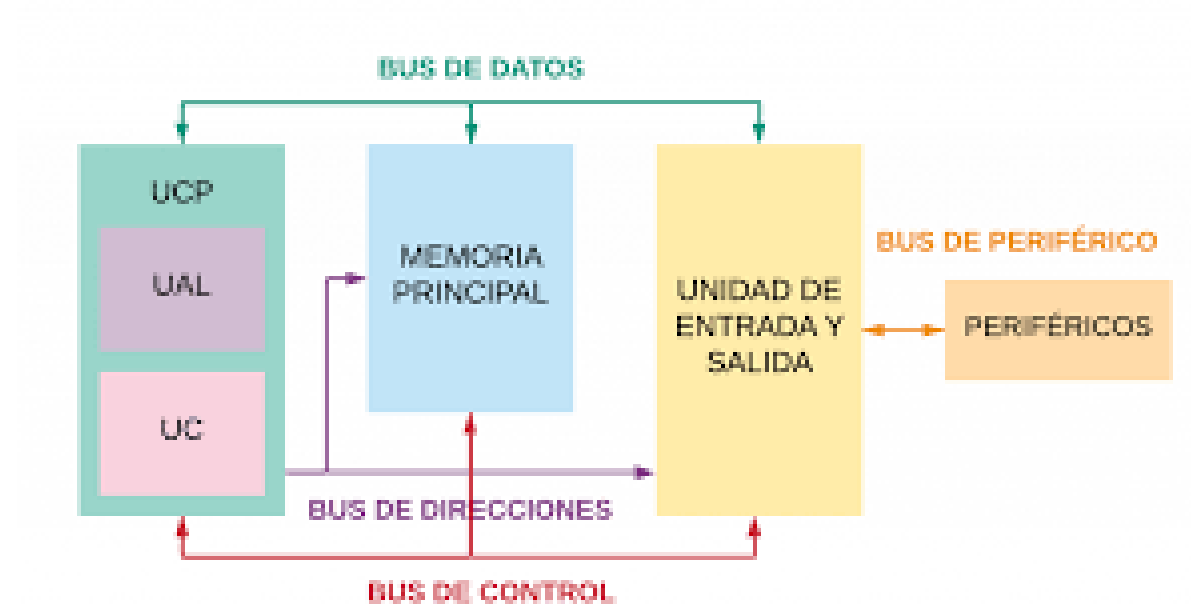
Gestión eficiente de la memoria: Permite que múltiples programas compartan la memoria física de forma segura y eficiente, incluso si la suma de su memoria requerida es mayor que la memoria física disponible.

Abstracción: La MMU oculta la complejidad de la gestión de memoria al sistema operativo y a las aplicaciones, lo que simplifica el desarrollo de software.

¿Cómo funciona?

- ❖ **Direcciones virtuales:** Los programas utilizan direcciones virtuales, que son números que representan las ubicaciones de memoria desde su punto de vista.
- ❖ **Tabla de páginas:** La MMU utiliza una tabla de páginas para mapear las direcciones virtuales a direcciones físicas. Esta tabla indica dónde se encuentra en la memoria física cada página de memoria virtual.
- ❖ **Traducción de direcciones:** Cuando un programa intenta acceder a una dirección de memoria, la MMU consulta la tabla de páginas para encontrar la dirección física correspondiente.
- ❖ **Fallo de página:** Si la página requerida no está en la memoria física, se produce un fallo de página. El sistema operativo se encarga de cargar la página faltante desde el disco duro a la memoria.

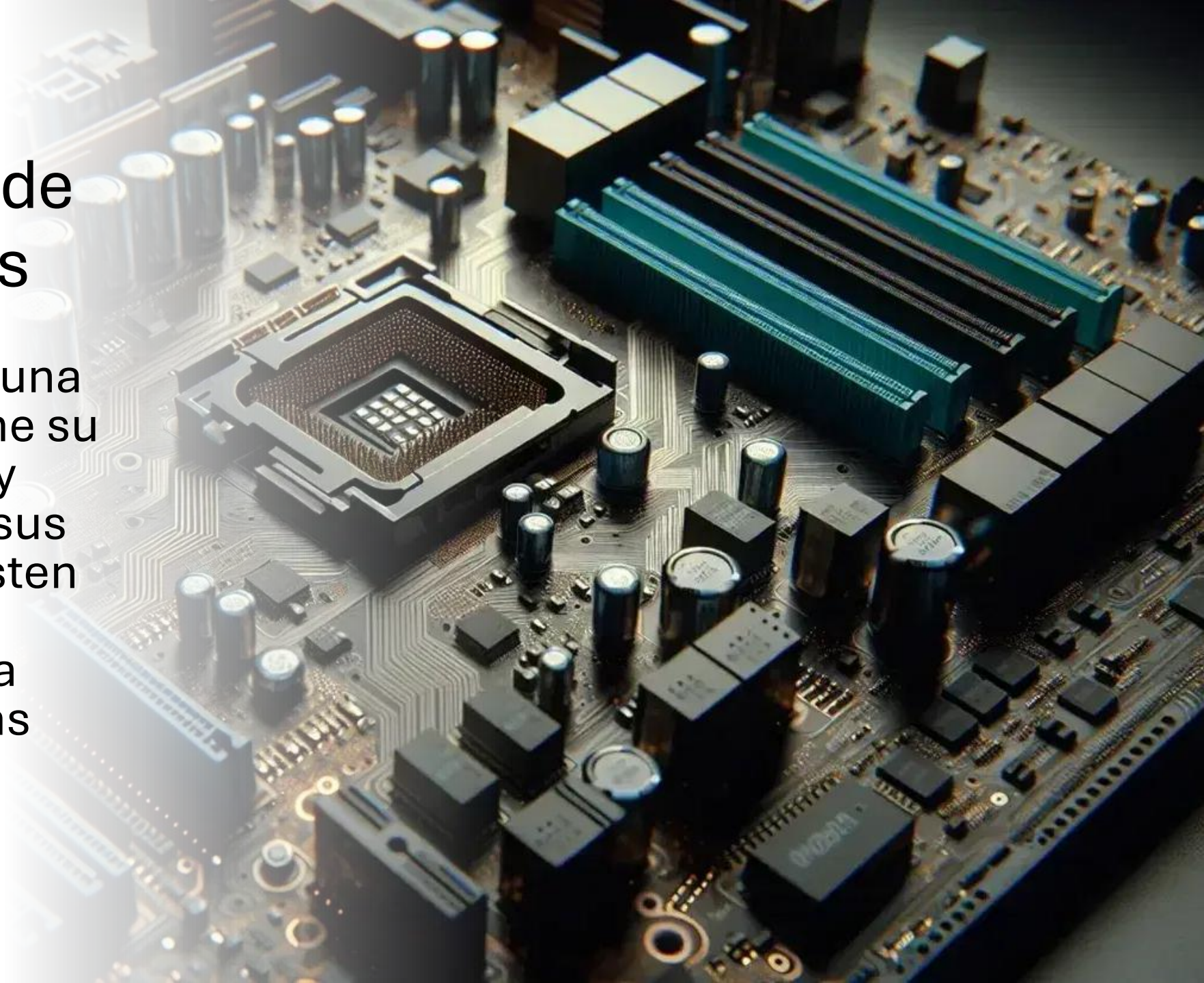


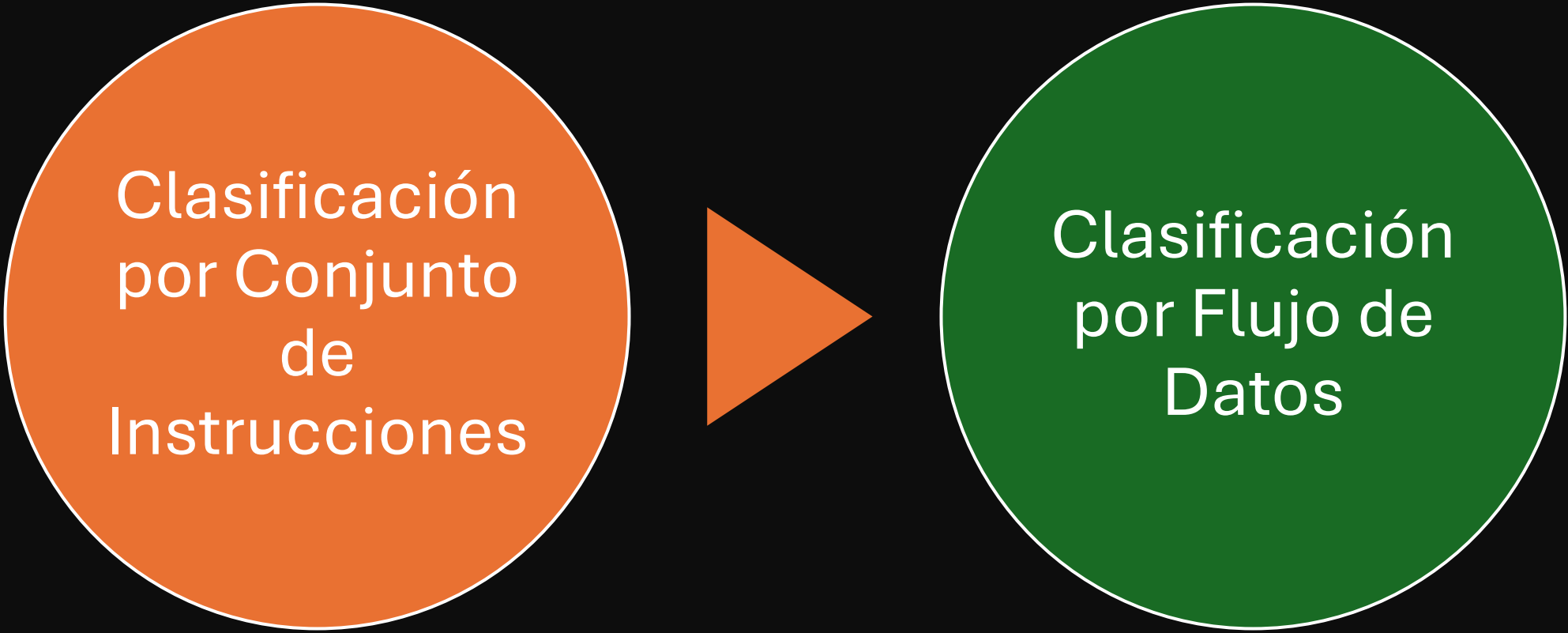


La arquitectura de un microprocesador dicta su rendimiento, eficiencia energética, y la compatibilidad con el software. Hay dos enfoques principales en su diseño: **CISC (Complex Instruction Set Computing)** y **RISC (Reduced Instruction Set Computing)**, que definen la complejidad y el número de instrucciones que un procesador puede manejar.

Tipos de Arquitecturas de Computadoras

La arquitectura de una computadora define su estructura interna y cómo interactúan sus componentes. Existen diversos tipos de arquitecturas, cada una con sus propias características y ventajas.



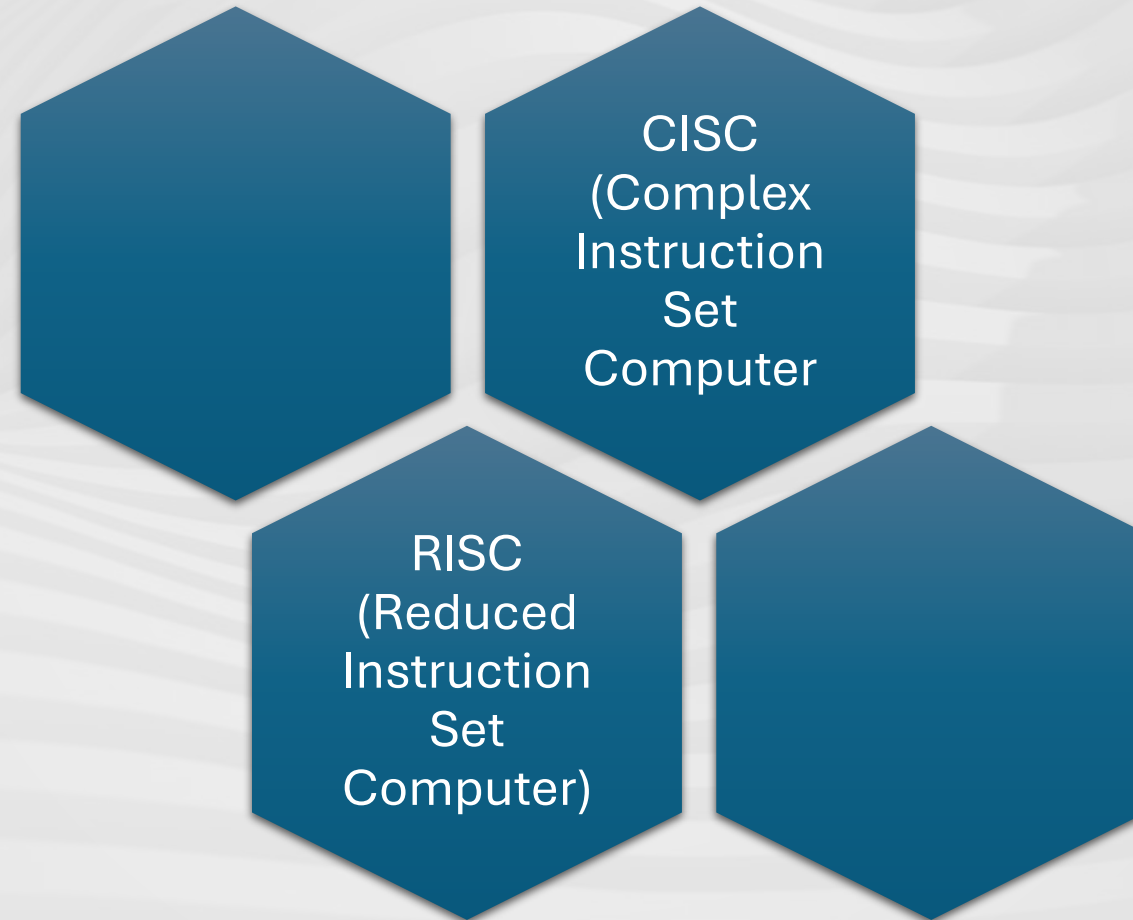


Clasificación
por Conjunto
de
Instrucciones

The diagram consists of two large circles, one orange on the left and one green on the right, connected by a large orange arrow pointing from left to right. The orange circle contains the text 'Clasificación por Conjunto de Instrucciones' and the green circle contains the text 'Clasificación por Flujo de Datos'. There are thin orange horizontal lines above and below the circles.

Clasificación
por Flujo de
Datos

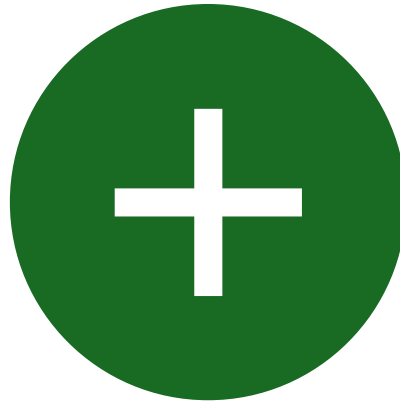
Clasificación por Conjunto de Instrucciones



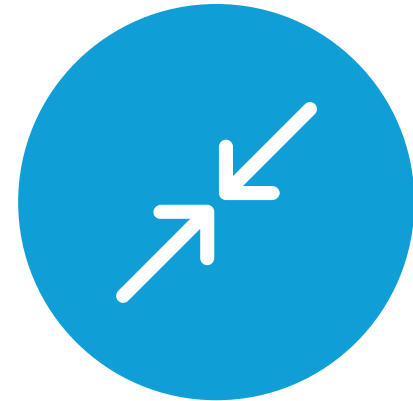
Clasificación por Flujo de Datos



SISD (SINGLE INSTRUCTION,
SINGLE DATA)



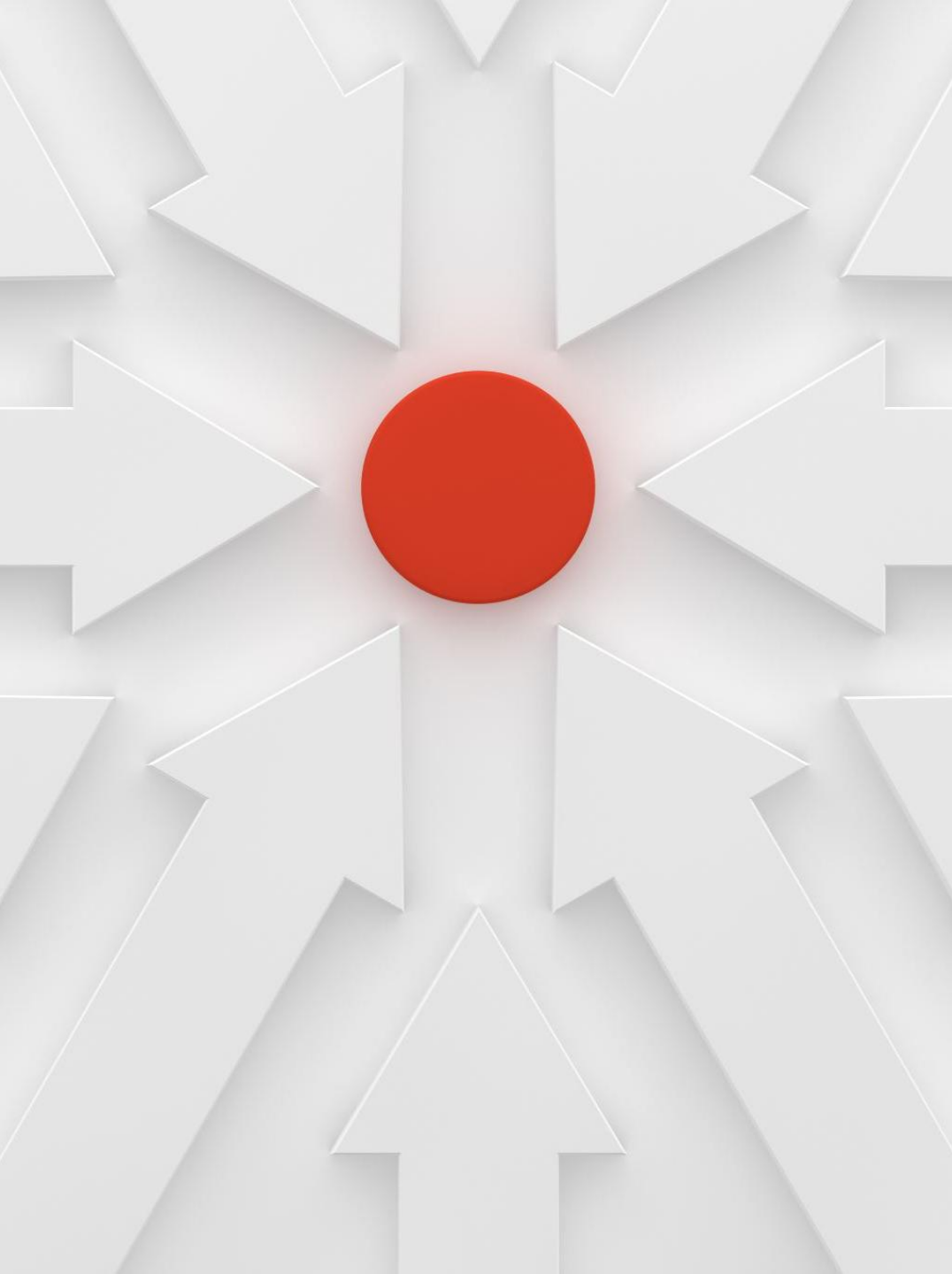
SIMD (SINGLE INSTRUCTION,
MULTIPLE DATA)



MIMD (MULTIPLE INSTRUCTION,
MULTIPLE DATA)

CISC (Complex Instruction Set Computer)

CISC, acrónimo de *Complex Instruction Set Computer* o Computadora con un Conjunto Complejo de Instrucciones, es un tipo de arquitectura de computadores que se caracteriza por tener un conjunto muy amplio y variado de instrucciones. Estas instrucciones pueden realizar operaciones complejas en una sola etapa, lo que en teoría hace que la programación sea más sencilla.

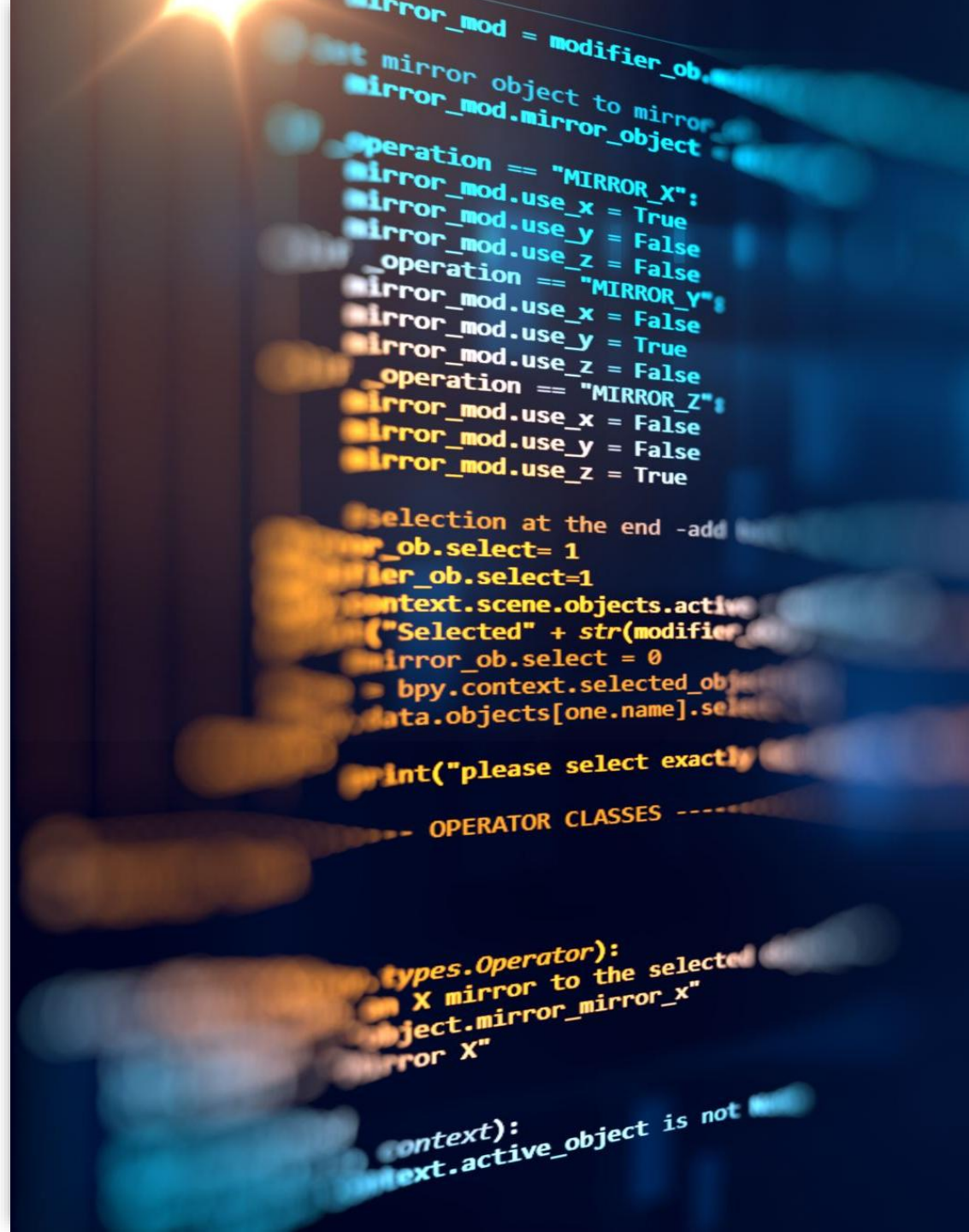


Características principales de CISC

- **Conjunto de instrucciones extenso:** Incluye instrucciones para realizar operaciones aritméticas, lógicas, de control de flujo y acceso a memoria, entre otras.
- **Instrucciones complejas:** Pueden realizar múltiples operaciones en un solo ciclo de reloj, lo que reduce el número total de instrucciones necesarias para ejecutar un programa.
- **Diseño orientado al software:** Se diseñaron para facilitar la programación a alto nivel, permitiendo que los compiladores generaran código más eficiente.

Ventajas de CISC

- ❖ **Facilidad de programación:** El conjunto amplio de instrucciones simplifica la tarea de escribir programas.
- ❖ **Mayor densidad de código:** Los programas CISC suelen ser más compactos debido a la complejidad de sus instrucciones.



Desventajas de CISC



1

Mayor complejidad del hardware: Los procesadores CISC son más complejos de diseñar y fabricar debido a la gran cantidad de instrucciones y microcódigo.



2

Menor rendimiento potencial: Las instrucciones complejas pueden requerir más ciclos de reloj para ejecutarse, lo que limita la velocidad máxima del procesador.



3

Dificultad de diseño de compiladores: La optimización de código para procesadores CISC es más compleja debido a la gran variedad de instrucciones disponibles.

RISC (Reduced Instruction Set Computer)

RISC, acrónimo de *Reduced Instruction Set Computer* o Computadora con un Conjunto Reducido de Instrucciones, es una arquitectura de procesadores que se caracteriza por utilizar un conjunto de instrucciones más simple y limitado en comparación con los procesadores CISC. Esta simplificación tiene como objetivo mejorar el rendimiento y la eficiencia energética.

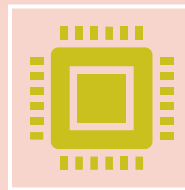
Características principales de RISC

- ❖ **Conjunto de instrucciones reducido:** Solo incluye las instrucciones más comunes y necesarias para ejecutar programas.
- ❖ **Instrucciones simples:** Cada instrucción realiza una operación básica, lo que facilita su decodificación y ejecución.
- ❖ **Ciclo de reloj único:** La mayoría de las instrucciones se ejecutan en un solo ciclo de reloj, lo que aumenta la velocidad del procesador.
- ❖ **Uso intensivo de registros:** Los datos se almacenan principalmente en registros internos del procesador, lo que reduce los accesos a la memoria y mejora el rendimiento.
- ❖ **Diseño orientado al hardware:** Los procesadores RISC se diseñan para aprovechar al máximo las características del hardware, como la canalización y el paralelismo.

Ventajas de RISC



Mayor rendimiento: La simplificación del conjunto de instrucciones permite una ejecución más rápida de los programas.



Menor consumo de energía: Los procesadores RISC son más eficientes energéticamente, lo que los hace ideales para dispositivos móviles y embebidos.



Diseño más sencillo: La arquitectura RISC es más fácil de diseñar y fabricar, lo que reduce los costos.



Mayor escalabilidad: Los procesadores RISC se pueden escalar fácilmente para ofrecer un mayor rendimiento.

Característica	CISC (Complex Instruction Set Computer)	RISC (Reduced Instruction Set Computer)
Conjunto de instrucciones	Gran variedad de instrucciones, incluyendo operaciones complejas como multiplicación y división.	Conjunto reducido de instrucciones simples, donde operaciones complejas se realizan mediante secuencias de instrucciones más básicas.
Diseño orientado a	Facilitar la programación, ofreciendo un conjunto amplio de instrucciones complejas. Sin embargo, esto puede llevar a un rendimiento menor y un mayor consumo de energía.	Maximizar el rendimiento y la eficiencia energética, utilizando un conjunto reducido de instrucciones simples. Esto permite diseños de hardware más sencillos y escalables.
Ciclo de reloj	Operaciones complejas pueden tomar más ciclos de reloj para completarse.	Operaciones simples se ejecutan en menos ciclos de reloj, mejorando el rendimiento.
Uso de registros	Puede haber un mayor uso de registros para almacenar datos y resultados intermedios.	Se prioriza el uso eficiente de registros, a menudo mediante el uso de registros temporales y al pipeline de instrucciones.
Complejidad	Mayor complejidad en el hardware debido a la gran variedad de instrucciones.	Diseño más modular y sencillo, lo que facilita la escalabilidad.
Rendimiento	Puede ser menor debido a la mayor complejidad y al mayor consumo de energía.	Mayor rendimiento por ciclo de reloj debido a la simplicidad de las instrucciones.
Consumo de energía	Puede ser mayor debido a la mayor complejidad y al mayor uso de recursos.	Mayor eficiencia energética debido al diseño más sencillo y al uso eficiente de recursos.
Densidad de código	Mayor densidad de código debido a la gran variedad de instrucciones.	Menor densidad de código debido al uso de secuencias de instrucciones más básicas para realizar tareas complejas.
Escalabilidad	Más difícil de escalar debido a la complejidad.	Más fácil de escalar, gracias a su diseño modular.
Ejemplos de procesadores	x86 (Intel, AMD)	ARM, PowerPC

¿Cuándo usar CISC y cuándo usar RISC?

CISC: Ideal para aplicaciones que requieren una alta densidad de código y una fácil programación, como sistemas operativos y algunas aplicaciones de escritorio.

RISC: Perfecto para dispositivos móviles, sistemas embebidos, servidores y cualquier aplicación que requiera un alto rendimiento y bajo consumo de energía.

Característica	SISD (Single Instruction, Single Data)	SIMD (Single Instruction, Multiple Data)	MIMD (Multiple Instruction, Multiple Data)
Descripción	Un solo procesador ejecuta una sola instrucción sobre un solo dato a la vez.	Un solo procesador ejecuta una misma instrucción sobre múltiples datos simultáneamente.	Múltiples procesadores ejecutan diferentes instrucciones sobre diferentes datos al mismo tiempo.
Ejemplo	Mayoría de las computadoras personales	Unidades de procesamiento gráfico (GPUs), procesadores vectoriales	Supercomputadoras, clústeres de computadoras
Aplicaciones típicas	Tareas generales, aplicaciones de oficina, programación secuencial	Procesamiento de imágenes, simulación, análisis numérico	Simulaciones a gran escala, inteligencia artificial, análisis de datos masivos
Ventajas	Simple, fácil de programar	Alto rendimiento para tareas altamente paralelas, bajo consumo energético por instrucción	Gran flexibilidad, alto rendimiento para tareas diversas
Desventajas	Bajo rendimiento para tareas paralelas	Menos flexible que MIMD, requiere datos estructurados de manera específica	Alto costo, complejidad de programación
Estructura de datos	Datos escalares	Datos vectoriales o matrices	Datos distribuidos

- **SISD:** Es la arquitectura más común en las computadoras personales. Cada instrucción se ejecuta secuencialmente sobre un solo dato. Es simple de programar, pero puede ser lenta para tareas que podrían beneficiarse del paralelismo.
- **SIMD:** Ideal para tareas que involucran la misma operación sobre grandes conjuntos de datos. Las GPUs son un ejemplo clásico, donde miles de núcleos procesan simultáneamente píxeles de una imagen.
- **MIMD:** Ofrece la máxima flexibilidad, permitiendo que múltiples procesadores trabajen de forma independiente en diferentes tareas. Se utiliza en supercomputadoras y clústeres para resolver problemas complejos que requieren gran cantidad de cómputo.



Cuando Usar Cada Arquitectura

- **SISD:** Para tareas generales y aplicaciones que no requieren un alto grado de paralelismo.
- **SIMD:** Para tareas que involucran grandes volúmenes de datos y operaciones repetitivas, como procesamiento de imágenes, video y simulación.
- **MIMD:** Para problemas complejos que requieren una gran cantidad de cómputo y que pueden descomponerse en tareas independientes, como modelado climático, análisis genómico y aprendizaje automático.

Tarea

- Realiza una investigación acerca de las arquitecturas SISD, SIMD y MIMD y genera un cuadro comparativo entre ellas con las siguientes características
 - Descripción
 - Ejemplo
 - Aplicaciones Típicas
 - Ventajas
 - Desventajas
 - Estructura de datos

